A lightweight canonicalization engine in C

William Zijie Zhang

Master's Thesis Defence, Polytechnique Montreal, Aug. 14, 2026

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Convex optimization

$$\begin{array}{ll} \text{minimize} & f_0(x) \\ \text{subject to} & f_i(x) \leq 0, \quad i = 1, \ldots, m \\ & Ax = b \end{array} \tag{1}$$

- variable $x \in \mathbf{R}^n$
- equality constraints are linear and $f_0, \ldots, f_m$ are convex
- can be solved globally and efficiently (both in theory and practice)

Functional form

The user writes the problem in a general functional form:

$$\begin{array}{ll} \text{minimize} & f_0(x_1, \ldots, x_k) \\ \text{subject to} & f_i(x_1, \ldots, x_k) \leq g_i(x_1, \ldots, x_k), \quad i = 1, \ldots, m_1 \\ & h_i(x_1, \ldots, x_k) = j_i(x_1, \ldots, x_k), \quad i = 1, \ldots, m_2 \end{array} \tag{2}$$

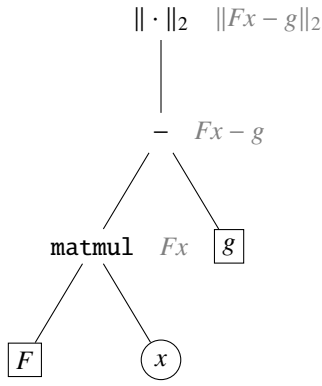
- encompasses any optimization problem
- potentially many variables of different sizes $x_1, \ldots, x_k$
- the functions are *expression trees* built from a library of *atoms*

example

$$\begin{array}{ll} \text{minimize} & \|Fx - g\|_2 + \mu \|x\|_2 \\ \text{subject to} & x \succeq 0 \end{array}$$

Expression trees

Internally the functional form is stored as **expression trees** — one per objective and constraint. Atoms are the nodes; variables, constants, and parameters are the leaves.



Disciplined convex programming (DCP)

- a grammar for specifying convex optimization problems [Grant et al., 2006]
- convex and concave functions are constructed by combining a set of **atoms**
- each atom has known curvature and monotonicity
- simple rules specify how atoms can be composed
- a problem that follows the DCP rules is convex-by-construction

Modeling languages implementing DCP

- Yalmip (MATLAB), CVX (MATLAB), CVXPY (Python), Convex.jl (Julia), CVXR (R)
- classic commercial software like AMPL and GAMS do not implement DCP

Modeling languages implementing DCP

- Yalmip (MATLAB), CVX (MATLAB), CVXPY (Python), Convex.jl (Julia), CVXR (R)
- classic commercial software like AMPL and GAMS do not implement DCP

- two main tasks of DCP-based modeling languages:
  - **analysis**: verify that a high level problem description follows the DCP rules
  - **synthesis**: rewrite it into an equivalent conic standard form:

$$\begin{array}{ll} \text{minimize} & c^\top x + d \\ \text{subject to} & b - Ax \in \mathcal{K} \end{array}$$

problem data A, b, c, d; $\mathcal{K}$ a product of standard cones

Functional conic form

Canonicalization first rewrites the problem into a *functional* conic form:

$$\begin{array}{ll} \text{minimize} & f_0(x_1, \ldots, x_k) \\ \text{subject to} & g_i(x_1, \ldots, x_k) \in \mathcal{K}_i, \quad i = 1, \ldots, m \end{array} \tag{3}$$

- f_0 and every g_i are now affine
- each $\mathcal{K}_i$ is a standard cone
- nonsmooth atoms are replaced by epigraph variables and cone constraints [Grant & Boyd, 2008]

least squares becomes:

$$\begin{array}{ll} \text{minimize} & t_1 + \mu\, t_2 \\ \text{s.t.} & x \in \mathbf{R}^n_+ \\ & (t_1,\, Fx - g) \in \mathcal{K}_{\text{soc}} \\ & (t_2,\, x) \in \mathcal{K}_{\text{soc}} \end{array}$$

Quadratic conic standard form

The backend materializes (**??**) into the numeric conic standard form:

$$\begin{array}{ll} \text{minimize} & \frac{1}{2}x^\top Px + c^\top x + d \\ \text{subject to} & b - Ax \in \mathcal{K} \end{array} \tag{4}$$

- data $P \succeq 0,\ A, b, c, d;\ \mathcal{K}$ a product of standard cones
- generalizes both QP ($P \neq 0, \mathcal{K} = \mathbf{R}^m_+$) and conic programs ($P = 0$)
- solvers: OSQP, HiGHS (QP); ECOS (conic); MOSEK, Clarabel (conic-quadratic)

least squares (**??**), $u = [\ x;\ t_1;\ t_2\ ]$ ($P = 0$):

$$\min\ c^\top u, \quad b - Au \in \mathcal{K}$$

$$c = \begin{bmatrix} 0 \\ 1 \\ \mu \end{bmatrix},\ A = \begin{bmatrix} -I & 0 & 0 \\ 0 & -1 & 0 \\ -F & 0 & 0 \\ 0 & 0 & -1 \\ -I & 0 & 0 \end{bmatrix},\ b = \begin{bmatrix} 0 \\ 0 \\ -g \\ 0 \\ 0 \end{bmatrix}$$

$$\mathcal{K} = \mathbf{R}^n_+ \times \mathcal{K}_{\text{soc}} \times \mathcal{K}_{\text{soc}}$$

a squared objective $\|Fx - g\|_2^2$ would instead populate P, with no epigraph variable t_1.

Summary

We have introduced four standard forms: the convex optimization standard form (**??**), the user's functional form (**??**), the functional conic form (**??**), and the numeric conic standard form (**??**).

A DCP modeling language works in two stages:

- the *frontend* takes the functional form (**??**), uses DCP to verify it is convex (**??**), and rewrites it into the functional conic form (**??**)
- the *backend* materializes (**??**) into the numeric data (**??**)

Summary

We have introduced four standard forms: the convex optimization standard form (**??**), the user's functional form (**??**), the functional conic form (**??**), and the numeric conic standard form (**??**).

A DCP modeling language works in two stages:

- the *frontend* takes the functional form (**??**), uses DCP to verify it is convex (**??**), and rewrites it into the functional conic form (**??**)
- the *backend* materializes (**??**) into the numeric data (**??**)

This thesis focuses on the *canonicalization backend* for large-scale and parametrized problems.

Outline

Background

Canonicalization

A lightweight C engine

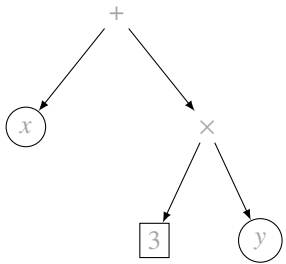
Results

Conclusions

Matrix stuffing: a worked example

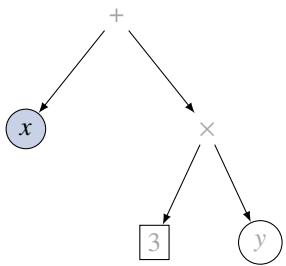
$x + 3y$ with $x, y, z \in \mathbf{R}^2$

row block of A



Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

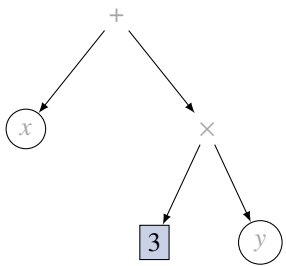


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

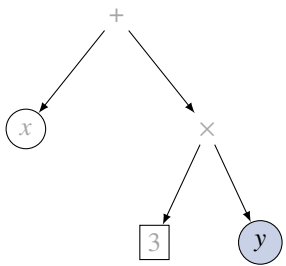


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

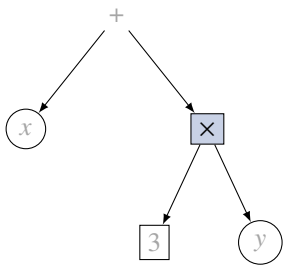


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)
- y : $\begin{bmatrix} 0 & I & 0 \end{bmatrix}$

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$

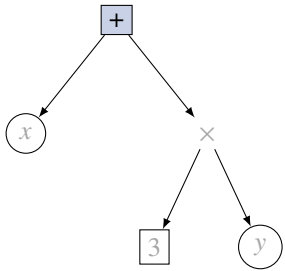


row block of A

- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)
- y : $\begin{bmatrix} 0 & I & 0 \end{bmatrix}$
- $3y$: $\begin{bmatrix} 0 & 3I & 0 \end{bmatrix}$

Matrix stuffing: a worked example

$x + 3y$ with $x, y, z \in \mathbf{R}^2$



row block of A

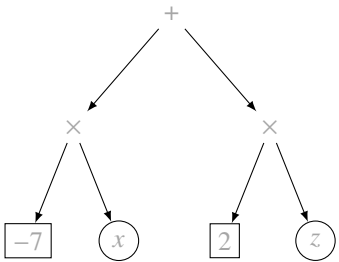
- x : $\begin{bmatrix} I & 0 & 0 \end{bmatrix}$
- 3 : $\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}$ (constant)
- y : $\begin{bmatrix} 0 & I & 0 \end{bmatrix}$
- $3y$: $\begin{bmatrix} 0 & 3I & 0 \end{bmatrix}$
- $x{+}3y$: $\begin{bmatrix} I & 3I & 0 \end{bmatrix}$

each block is 2×2; the z block stays 0. This is row block 1 of A.

Stacking constraints

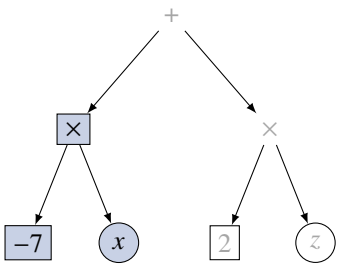
$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

row block of A



Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

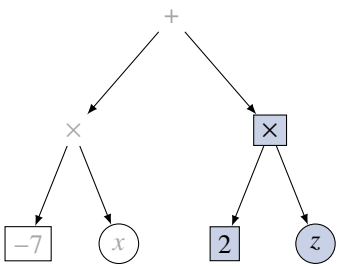


row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$

Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

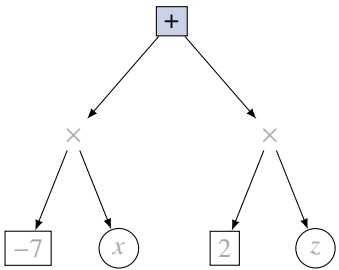


row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$
- $2z$: $\begin{bmatrix} 0 & 0 & 2I \end{bmatrix}$

Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$

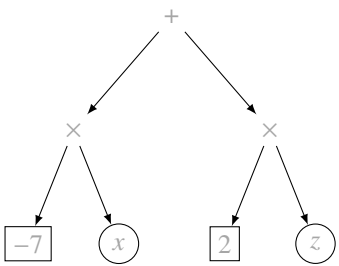


row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$
- $2z$: $\begin{bmatrix} 0 & 0 & 2I \end{bmatrix}$
- $-7x+2z$: $\begin{bmatrix} -7I & 0 & 2I \end{bmatrix}$

Stacking constraints

$-7x + 2z$ with $x, y, z \in \mathbf{R}^2$



row block of A

- $-7x$: $\begin{bmatrix} -7I & 0 & 0 \end{bmatrix}$
- $2z$: $\begin{bmatrix} 0 & 0 & 2I \end{bmatrix}$
- $-7x+2z$: $\begin{bmatrix} -7I & 0 & 2I \end{bmatrix}$

stacking both constraints:

$$A = \begin{bmatrix} I & 3I & 0 \\ -7I & 0 & 2I \end{bmatrix} \in \mathbf{R}^{4\times 6}$$

Variable offsets

- the backend stacks all variables into one vector u
- each variable has a unique **id**; `id_to_col` maps it to a **column offset**
- a variable of size n occupies n consecutive columns of A

$$x \in \mathbf{R}^2, \quad y \in \mathbf{R}^3 \qquad \Rightarrow \qquad u = \big[\ \underbrace{x_1\ x_2}_{\text{cols } 1\text{–}2}\ \ \underbrace{y_1\ y_2\ y_3}_{\text{cols } 3\text{–}5}\ \big]^\top \in \mathbf{R}^5$$

$$\texttt{id_to_col} = \{\, \mathrm{id}(x) : 0,\ \ \mathrm{id}(y) : 2 \,\}$$

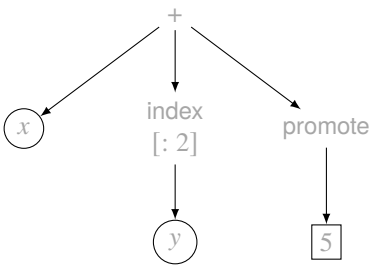
- so A has 5 columns (`var_length`); a final column holds the constant b

Building the coefficient blocks

$$x + y_{[:2]} + 5$$

$$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$$

each node holds {var id ↦ `scipy.sparse`}

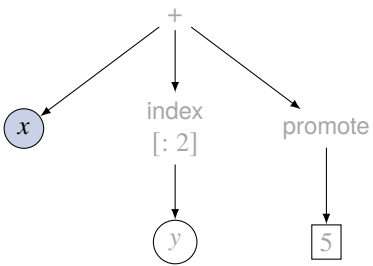


`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys.
Constant id = -1.

Building the coefficient blocks

$$x + y_{[:2]} + 5$$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

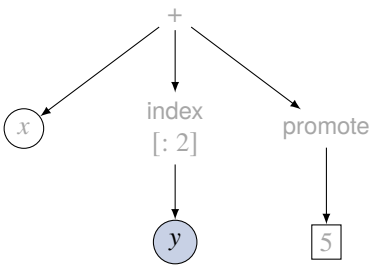
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

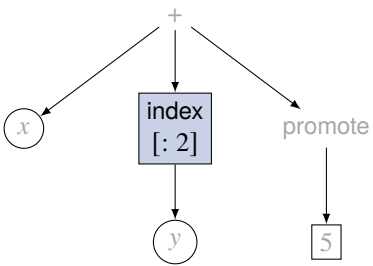
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$
- y : $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Building the coefficient blocks

$$x + y_{[:2]} + 5$$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

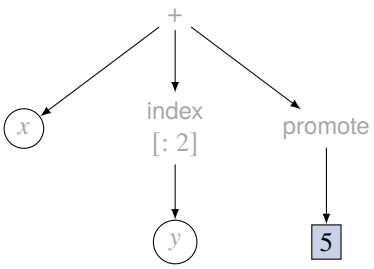
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$
- y : $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]\}$
- $y_{[:2]}$: $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\}$ (rows 0, 1)

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys.
Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

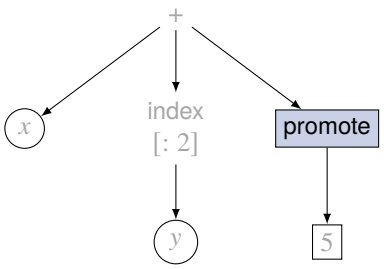
- x : $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\}$
- y : $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]\}$
- $y_{[:2]}$: $\{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\}$ (rows 0, 1)
- 5 : $\{-1 \mapsto [5]\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

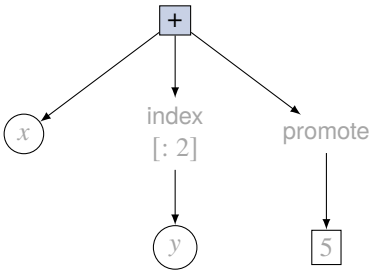
- x : $\{x \mapsto \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\}$
- y : $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\}$
- $y_{[:2]}$: $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}\}$ (rows 0, 1)
- 5 : $\{-1 \mapsto [\,5\,]\}$
- promote: $\{-1 \mapsto \begin{bmatrix} 5 \\ 5 \end{bmatrix}\}$ (repeat row)

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys.
Constant id = -1.

Building the coefficient blocks

$x + y_{[:2]} + 5$

$x \in \mathbf{R}^2,\ y \in \mathbf{R}^3$



each node holds {var id ↦ `scipy.sparse`}

- x : $\{x \mapsto \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}\}$
- y : $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\}$
- $y_{[:2]}$: $\{y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}\}$ (rows 0, 1)
- 5 : $\{-1 \mapsto [5]\}$
- promote: $\{-1 \mapsto \begin{bmatrix} 5 \\ 5 \end{bmatrix}\}$ (repeat row)
- $+$: $\left\{ \begin{matrix} x \mapsto \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix},\ y \mapsto \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \\ -1 \mapsto \begin{bmatrix} 5 \\ 5 \end{bmatrix} \end{matrix} \right\}$

`scipy.sparse`: variable → `eye_array` (CSC); `index` selects rows; `promote` repeats a row; `sum` merges per-variable keys. Constant id = -1.

Assembling $[A\ b]$

`id_to_col` places each variable's block at its column offset; the constant fills the last column:

- $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\} \to$ columns 1–2; $\quad \{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\} \to$ columns 3–5

CVXPY: `flatten_tensor` collapses (row, col) into one sparse $[A\ b]$; `id_to_col[-1] = var_length` reserves the constant column.

Assembling $[A\ b]$

`id_to_col` places each variable's block at its column offset; the constant fills the last column:

- $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\} \to$ columns 1–2; $\quad \{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\} \to$ columns 3–5
-

$$[A\ b] = \left[\begin{array}{cc|ccc|c} 1 & 0 & 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 1 & 0 & 5 \end{array}\right] \in \mathbf{R}^{2\times 6}$$

CVXPY: `flatten_tensor` collapses (row, col) into one sparse $[A\ b]$; `id_to_col`$[-1]$ = `var_length` reserves the constant column.

Assembling $[A\ b]$

`id_to_col` places each variable's block at its column offset; the constant fills the last column:

- $\{x \mapsto \left[\begin{smallmatrix} 1 & 0 \\ 0 & 1 \end{smallmatrix}\right]\} \to$ columns 1–2; $\quad \{y \mapsto \left[\begin{smallmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{smallmatrix}\right]\} \to$ columns 3–5
- $$[A\ b] = \left[\begin{array}{cc|ccc|c} 1 & 0 & 1 & 0 & 0 & 5 \\ 0 & 1 & 0 & 1 & 0 & 5 \end{array}\right] \in \mathbf{R}^{2\times 6}$$
- the y_3 column is all zeros — $y_{[:2]}$ never touched it

CVXPY: `flatten_tensor` collapses (row, col) into one sparse $[A\ b]$; `id_to_col[-1] = var_length` reserves the constant column.

Parameters, path 1: `ignore_dpp`

- prepends a reduction called `EvalParams`: every parameter node in the expression trees is replaced by a **constant** holding its current value
- canonicalization then runs on a parameter-free problem

$$\min\ t_1 + \mu\, t_2 \xrightarrow{\texttt{EvalParams: } (\mu,\, g) \leftarrow (2.5,\, \hat{g})} \min\ t_1 + 2.5\, t_2 \qquad \Rightarrow \qquad c = \begin{bmatrix} 0 \\ 1 \\ 2.5 \end{bmatrix}$$

Parameters, path 1: `ignore_dpp`

- prepends a reduction called `EvalParams`: every parameter node in the expression trees is replaced by a **constant** holding its current value
- canonicalization then runs on a parameter-free problem

$$\min\ t_1 + \mu\, t_2 \xrightarrow{\texttt{EvalParams}:\ (\mu,\, g) \leftarrow (2.5,\, \hat{g})} \min\ t_1 + 2.5\, t_2 \qquad \Rightarrow \qquad c = \begin{bmatrix} 0 \\ 1 \\ 2.5 \end{bmatrix}$$

- the numeric data drop out directly — but the **full reduction chain re-runs** every time a parameter changes

Parameters, path 2: the DPP parameter map

- if the problem follows the DPP ruleset, CVXPY compiles **once** and caches a linear map from parameters to problem data
- for the objective: $[c;\, d] = Q\,\tilde{\theta}$ with $\tilde{\theta} = (g, \mu, 1)$

$$Q = \begin{array}{c} \\ x \\ t_1 \\ t_2 \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^\top & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 0 \\ 1 \\ \mu \\ 0 \end{bmatrix}$$

Parameters, path 2: the DPP parameter map

- if the problem follows the DPP ruleset, CVXPY compiles **once** and caches a linear map from parameters to problem data
- for the objective: $[c;\, d] = Q\,\tilde{\theta}$ with $\tilde{\theta} = (g, \mu, 1)$

$$Q = \begin{array}{c} \\ x \\ t_1 \\ t_2 \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^\top & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 0 \\ 1 \\ \mu \\ 0 \end{bmatrix}$$

- re-solve = one sparse matvec, the reduction chain is bypassed entirely
- but: requires DPP compliance, and the cached map must be stored

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

The key insight

- for any parameterized problem, under mild conditions the **sparsity pattern** of $A(\theta)$ is fixed across the family
- only the **numerical values** of the nonzeros change with θ

The key insight

- for any parameterized problem, under mild conditions the **sparsity pattern** of $A(\theta)$ is fixed across the family
- only the **numerical values** of the nonzeros change with θ
- reframe matrix stuffing as **sparse Jacobian evaluation**:
  - symbolic phase: compute the sparsity pattern of $\partial A / \partial \theta$ once
  - numeric phase: refill values on every solve
- no restriction on how parameters enter the problem

Two-phase design

Init phase (run once per problem family):

- allocate one `expr` struct per tree node
- traverse the tree to compute sparsity patterns
- allocate Jacobians as CSR matrices

Eval phase (run on every solve, with new θ):

- walk the tree, fill numerical values into pre-allocated buffers
- stuff the resulting A, b, c into the solver

Two-phase design

Init phase (run once per problem family):

- allocate one `expr` struct per tree node
- traverse the tree to compute sparsity patterns
- allocate Jacobians as CSR matrices

Eval phase (run on every solve, with new θ):

- walk the tree, fill numerical values into pre-allocated buffers
- stuff the resulting A, b, c into the solver

Cost model: init is $O(\text{tree size})$ once; eval is $O(\text{nnz}(A))$ per solve.

The two phases on our example

replace the constant 3 with a parameter θ: $\quad x + \theta y, \quad x, y, z \in \mathbf{R}^2$

Init (once per problem family)

- walk the tree, record *which* blocks are nonzero
- pattern: blocks x, y (z empty)
- allocate the CSR matrix $\begin{bmatrix} \bullet & \bullet & 0 \end{bmatrix}$

Eval (every solve, new θ)

- refill values into the fixed pattern
- θ=3 : $\begin{bmatrix} I & 3I & 0 \end{bmatrix}$
- θ=7 : $\begin{bmatrix} I & 7I & 0 \end{bmatrix}$
- structure never recomputed

The two phases on our example

replace the constant 3 with a parameter θ: $\quad x + \theta y, \quad x, y, z \in \mathbf{R}^2$

Init (once per problem family)

- walk the tree, record *which* blocks are nonzero
- pattern: blocks x, y (z empty)
- allocate the CSR matrix $\begin{bmatrix} \bullet & \bullet & 0 \end{bmatrix}$

Eval (every solve, new θ)

- refill values into the fixed pattern
- θ=3 : $\begin{bmatrix} I & 3I & 0 \end{bmatrix}$
- θ=7 : $\begin{bmatrix} I & 7I & 0 \end{bmatrix}$
- structure never recomputed

the current backend redoes the **whole** left column on every solve; the engine pays it once.

(quadratic objectives add a matrix P, built the same way from second-order coefficients.)

The expr struct

```c
typedef struct expr {
    int     d1, d2, size;    /* shape (d1 x d2) */
    int     n_vars;
    int     var_id;          /* offset, or NOT_A_VARIABLE */

    struct expr *left, *right;

    double     *value;       /* forward-pass output */
    CSR_Matrix *jacobian;    /* size x n_vars sparse */

    forward_fn       forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
} expr;
```

The atom contract

Every atom implements three functions:

Function	Phase / Purpose
`forward`	numeric: compute the atom's output value
`jacobian_init`	symbolic: allocate sparsity pattern of the Jacobian
`eval_jacobian`	numeric: fill Jacobian values from children

- `jacobian_init` is called **once** during the init phase
- `eval_jacobian` is called on **every** solve

Atom example: elementwise multiply

- operation: $z = a \odot b$ (elementwise product of two expressions)
- Jacobian: $\partial z/\partial x = \mathrm{diag}(a)\,\partial b/\partial x + \mathrm{diag}(b)\,\partial a/\partial x$

Atom example: elementwise multiply

- operation: $z = a \odot b$ (elementwise product of two expressions)
- Jacobian: $\partial z/\partial x = \operatorname{diag}(a)\,\partial b/\partial x + \operatorname{diag}(b)\,\partial a/\partial x$
- `jacobian_init`: union of the sparsity patterns of $\partial a/\partial x$ and $\partial b/\partial x$
- `eval_jacobian`: scale child Jacobian rows by the current values of a and b

Atom example: elementwise multiply

- operation: $z = a \odot b$ (elementwise product of two expressions)
- Jacobian: $\partial z/\partial x = \mathrm{diag}(a)\,\partial b/\partial x + \mathrm{diag}(b)\,\partial a/\partial x$
- `jacobian_init`: union of the sparsity patterns of $\partial a/\partial x$ and $\partial b/\partial x$
- `eval_jacobian`: scale child Jacobian rows by the current values of a and b
- key observation: the sparsity union is computed **once**; per-solve work is just row scaling
- this is where **parameter values** enter without changing structure

Atom example: matrix multiplication

- operation: $z = A \cdot y$ (where A is a constant or parameter, y is an expression)
- Jacobian: $\partial z / \partial x = A \cdot \partial y / \partial x$

Atom example: matrix multiplication

- operation: $z = A \cdot y$ (where A is a constant or parameter, y is an expression)
- Jacobian: $\partial z / \partial x = A \cdot \partial y / \partial x$
- `jacobian_init`: sparsity pattern of $A \cdot J_y$ derived from the sparsity of A and J_y
- `eval_jacobian`: sparse-sparse multiply, writing into pre-allocated CSR buffers
- the structural pattern — which entries are nonzero — is fixed once A's sparsity is known

Assembling A

- each constraint contributes a row block of A
- after `eval_jacobian` runs on every node, the root Jacobians *are* the rows of A
- we copy them into a flat CSR buffer in the layout the solver expects

Assembling A

- each constraint contributes a row block of A
- after `eval_jacobian` runs on every node, the root Jacobians *are* the rows of A
- we copy them into a flat CSR buffer in the layout the solver expects
- the column structure of A is computed once; row values are refilled per solve
- no Python in the hot path; no allocation in the eval phase

What “lightweight” means

- pure C; no Python in the inner loop
- no symbolic graph mutation between solves
- CSR storage throughout; cache-friendly traversals
- no general-purpose AD framework (no JAX, no Autograd) — we only implement the atoms we need

What “lightweight” means

- pure C; no Python in the inner loop
- no symbolic graph mutation between solves
- CSR storage throughout; cache-friendly traversals
- no general-purpose AD framework (no JAX, no Autograd) — we only implement the atoms we need
- contrast with today’s backend:
  - no 3D parameter tensor in memory
  - sparsity computed once, not on every solve
  - no restriction on how parameters enter

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Benchmark setup

- hardware: laptop (TODO: spec)
- software: CVXPY pinned (TODO: version)
- baselines:
  - CVXPY DPP path (default)
  - CVXPY `ignore_dpp=True`
  - our C engine
- metrics:
  - cold-start time (first solve)
  - warm-restart time (subsequent solves, new θ)
  - peak memory

Problem suite

- **standard benchmarks**
  - LASSO, portfolio optimization, model-predictive control
  - representative of real-world use cases
- **stress tests**
  - large sparse problems, deep expression trees
  - designed to expose canonicalization overhead
- **least squares**
  - bread-and-butter case; well-understood baseline

Hypotheses (no numbers yet)

- **cold-start time:** comparable to ignore_DPP (no caching benefit on first solve)
- **warm-restart time:** approaches the DPP path's amortized cost, without the DPP rule restriction
- **peak memory:** below DPP, because no 3D parameter tensor is materialized

Hypotheses (no numbers yet)

- **cold-start time:** comparable to ignore_DPP (no caching benefit on first solve)
- **warm-restart time:** approaches the DPP path's amortized cost, without the DPP rule restriction
- **peak memory:** below DPP, because no 3D parameter tensor is materialized
- **break-even:** after ~ 2–3 solves on the same problem family, our engine should beat ignore_DPP

Results

TODO: insert benchmark plots + tables here

- runtime comparison (cold + warm)
- memory comparison
- scaling with problem size

Outline

Background

Canonicalization

A lightweight C engine

Results

Conclusions

Summary

- reframed CVXPY matrix stuffing as **sparse Jacobian evaluation**
- built a lightweight **C engine** that:
  - computes sparsity once per problem family
  - refills values per solve
  - avoids the 3D parameter tensor of the DPP path
- works for parameterized problems **without the DPP rule restriction**

Summary

- reframed CVXPY matrix stuffing as **sparse Jacobian evaluation**
- built a lightweight **C engine** that:
  - computes sparsity once per problem family
  - refills values per solve
  - avoids the 3D parameter tensor of the DPP path
- works for parameterized problems **without the DPP rule restriction**
- (pending) benchmark wins over ignore_DPP, competitive with DPP

Limitations and future work

Limitations

- partial atom coverage — not yet a drop-in replacement for CVXPY
- C library, not yet integrated as a CVXPY backend
- single-threaded; no GPU

Future work

- expand atom library to cover all of CVXPY's standard atoms
- ship as a Python-importable CVXPY backend
- explore non-DPP parameterizations where sparsity is still stable

Thank you

Questions?